

Python Programming

A Practical Introduction

Hoa T. Vu

Variables

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10          # int
pi = 3.14       # float
name = "Alice"  # str
flag = True     # bool
```

```
print(x)        # 10
print(type(x))  # <class 'int'>

x = x + 1       # reassign
x += 1          # shorthand
```

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10                # int                print(x)                # 10
pi = 3.14             # float              print(type(x))          # <class 'int'>
name = "Alice"        # str
flag = True           # bool

x = x + 1             # reassign
x += 1                # shorthand
```

- No declaration needed, just assign
- Types can change: `x = 10` then `x = "hello"` is valid

Common Operations

Arithmetic

```
7 + 2    # 9
7 - 2    # 5
7 * 2    # 14
7 / 2    # 3.5    (float)
7 // 2   # 3      (floor div)
7 % 2    # 1      (remainder)
7 ** 2   # 49     (power)
```

Strings

```
s = "hello"
len(s)                # 5
s.upper()             # "HELLO"
s[0]                  # "h"
s[1:4]                # "ell"

# f-string
n = 42
f"answer: {n}"        # "answer: 42"
```

Lists

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]          # 3          (first element)
```

```
nums[-1]         # 5          (last element)
```

```
nums[1:3]        # [1, 4]    (slice)
```

```
len(nums)        # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]
```

```
nums[0]          # 3          (first element)
```

```
nums[-1]         # 5          (last element)
```

```
nums[1:3]        # [1, 4]    (slice)
```

```
len(nums)        # 5
```

```
nums.append(9)    # [3, 1, 4, 1, 5, 9]
```

```
nums.pop()        # removes and returns 9
```

```
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists can hold mixed types: [1, "hello", 3.14, True]

List Comprehension

Build a list in one line more readable than a loop.

```
# squares of 0..4  
squares = [x**2 for x in range(5)]  
# [0, 1, 4, 9, 16]
```

```
# even numbers only  
evens = [x for x in range(10) if x % 2 == 0]  
# [0, 2, 4, 6, 8]
```

```
# nested: 2-D grid of (row, col) pairs  
pairs = [(r, c) for r in range(3) for c in range(3)]
```

Dictionaries

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student         # True  
student.get("phone", "N/A") # "N/A" if missing
```

Dictionaries

A mapping from **keys** to **values**.

```
student = {  
    "name": "Alice",  
    "age": 22,  
    "gpa": 3.8,  
}
```

```
student["name"]          # "Alice"  
student["age"] = 23      # update  
student["major"] = "CS"  # add new key
```

```
"gpa" in student          # True  
student.get("phone", "N/A") # "N/A" if missing
```

Keys must be *immutable* (strings, numbers, tuples). Values can be anything.

Iterating Dictionaries

```
grades = {"Alice": 90, "Bob": 85, "Carol": 92}
```

```
# keys
```

```
for name in grades:  
    print(name)
```

```
# values
```

```
for score in grades.values():  
    print(score)
```

```
# both
```

```
for name, score in grades.items():  
    print(f"{name}: {score}")
```

Conditionals

if / elif / else

Python uses **indentation** (4 spaces) to define blocks; no braces.

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C"
```

Comparison Operators

- `==` equal `!=` not equal
- `<` `<=` `>` `>=` ordering
- `and` `or` `not` logical
- `in` membership test

```
x = 5
x > 3 and x < 10    # True
x in [1, 3, 5, 7]   # True
not (x == 4)        # True
```

Loops

for Loop

iterate over a list

```
for x in [2, 4, 6]:  
    print(x)
```

iterate with index

```
for i, x in enumerate([2, 4, 6]):  
    print(i, x)
```

range(start, stop, step)

```
for i in range(5):                # 0, 1, 2, 3, 4  
    print(i)
```

```
for i in range(0, 10, 2):        # 0, 2, 4, 6, 8  
    print(i)
```

while Loop

```
n = 1
while n < 100:
    n *= 2
print(n)    # 128
```

break and continue

```
for x in range(10):
    if x == 5:
        break           # exit the loop entirely
    if x % 2 == 0:
        continue        # skip to next iteration
    print(x)             # prints 1 3
```

- Python is widely used for data science and machine learning development.

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)

- Python is widely used for data science and machine learning development.
- Python can be slow, but a lot of underlying libraries were written in fast languages (e.g., C++)
- You can make Python code faster using Cython.

Simple Examples

Example: Prime Check

Is n prime? Check divisors from 2 to $\lfloor \sqrt{n} \rfloor$.

```
import math

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, math.isqrt(n) + 1):
        if n % i == 0:
            return False
    return True

print(is_prime(7))    # True
print(is_prime(12))   # False
```

Example: Word Counts

Count how many times each word appears in a string.

```
text = "the cat sat on the mat the cat"

counts = {}
for word in text.split():
    counts[word] = counts.get(word, 0) + 1

print(counts)
# {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1}
```

Example: Palindrome Check

A palindrome reads the same forwards and backwards.

```
def is_palindrome(s):  
    s = s.lower()  
    left, right = 0, len(s) - 1  
    while left < right:  
        if s[left] != s[right]:  
            return False  
        left += 1  
        right -= 1  
    return True  
  
print(is_palindrome("racecar"))    # True  
print(is_palindrome("hello"))     # False
```